Introduction to Deep Learning

Dr. Öğr. Üyesi Ümit Demirbaga

Deep Learning Final Project

ANN, Autoencoder, and CNN Applications

Student Name: Rayan Dughmoush
Student Number: 2221251391
Project Language: English
Submission Date: 07.06.2026

Source code: https://github.com/raydughmsh/DLFinalProject

Contents

1 Abstract **3**

2 Introduction **4**

3 Selected Models and Applications **5**

3.1 Model 1 - Artificial Neural Network: SMS Spam Detection 5

3.2 Model 2 - Autoencoder: MNIST Image Reconstruction 5

3.3 Model 3 - CNN: Fashion-MNIST Classification 5

4 Datasets **6**

4.1 SMS Spam Collection (ANN) 6

4.2 MNIST Handwritten Digits (Autoencoder) 6

4.3 Fashion-MNIST (CNN) 7

5 Model Architectures **8**

5.1 ANN Architecture 8

5.2 Autoencoder Architecture 8

5.3 CNN Architecture 9

6 Training Process **10**

6.1 ANN Training 10

6.2 Autoencoder Training 10

6.3 CNN Training 10

7 Experimental Results **11**

7.1 ANN - SMS Spam Detection 11

7.2 Autoencoder - MNIST Reconstruction 11

7.3 CNN - Fashion-MNIST Classification 12

8 Graphs and Commentary **13**

8.1 ANN - Training Curves 13

8.2 Autoencoder - Reconstruction Results 14

8.3 CNN - Training Curves and Evaluation 15

9 Discussion **17**

9.1 Was the model successful? 17

9.2 Did loss decrease during training? 17

9.3 Was validation performance close to training performance? 17

9.4 Was overfitting observed? 17

9.5 Strengths and Weaknesses 18
9.6 What could be improved? 18

10 Conclusion **19**

1 Abstract

This project involves the design and evaluation of three deep learning models which have been created in order to satisfy the requirements of the final project in the Introduction to Deep Learning course. The three models chosen are: (1) an Artificial Neural Network (ANN) that implements binary text classification for SMS spam detection; (2) a Dense Autoencoder that solves the problem of image reconstruction in unsupervised learning by using the MNIST handwritten digit database; and (3) a Convolutional Neural Network (CNN) that implements multi-class image classification based on the Fashion-MNIST clothing database. All the three models have been coded in Python, using TensorFlow/Keras library. ANN obtained a test accuracy of 97.49%; Autoencoder obtained a test reconstruction MSE of 0.0087; CNN obtained a test accuracy of 91%.

2 Introduction

Deep learning is a branch of machine learning where multi-layered neural networks are used for hierarchical data representation learning. In the past decade, deep learning has set state-of-the-art performance in a wide variety of fields such as natural language processing, computer vision, speech recognition, and generation modelling. In this project, three different models of deep learning are implemented based on varying problems and data:

• **Text Data:** Here an Artificial Neural Network (ANN) is applied for classifying SMS messages into categories like spam and ham (non-spam). In this problem, after text to vector conversion using TF-IDF vectorization, the ANN learns the classification of the data.

• **Image Data (Unsupervised Learning):** The Dense Autoencoder model is used for compression and reconstruction of 28x28 grayscale images from the MNIST data set.

This is an unsupervised task as there is no classification or labeling of the data and the Autoencoder learns a compressed 32-dimensional representation of the data.

• **Image Data (Supervised Learning):** The CNN model is used to classify 28 x 28 grayscale images from the Fashion-MNIST data set into 10 clothing categories.

The source code for all three models is publicly available on GitHub:
https://github.com/raydughmsh/DLFinalProject

All models were implemented in Python 3 using TensorFlow 2.20.0, scikit-learn, NumPy, Matplotlib, seaborn, and pandas.

3 Selected Models and Applications

3.1 Model 1 - Artificial Neural Network: SMS Spam Detection

Aim: Classify SMS messages as either spam (1) or ham/legitimate (0).

Data type: Text data (raw English SMS messages).

Problem type: Binary classification.

Why ANN? After transforming raw text into a numerical vector through TF-IDF technique, this will make the problem a simple table-based classification problem. An ANN can handle non-linear relationships in multi-dimensional space that are created by TF-IDF vectors. While both RNNs and CNNs need the data to be in sequence form, ANNs can learn from text data regardless of sequence order.

3.2 Model 2 - Autoencoder: MNIST Image Reconstruction

Aim: Learn a compact representation of handwritten digit images and reconstruct the original images from it.

Data type: Image data (28×28 grayscale images, digits 0-9).

Problem type: Unsupervised representation learning / image reconstruction.

Why Autoencoder? The Autoencoder is especially made to obtain low-dimensional representations using an encoder-decoder framework based on reconstruction loss. There is no need for any classes or labels. The MNIST dataset has clean standardized images which make it the perfect dataset to test out the performance of reconstruction.

3.3 Model 3 - CNN: Fashion-MNIST Classification

Aim: Classify 28×28 grayscale images of clothing into one of 10 categories.

Data type: Image data (28×28 grayscale fashion item images).

Problem type: Multi-class image classification (10 classes).

Why CNN? CNNs leverage the spatial dependency among pixels with the use of filters that can recognize features regardless of where those features appear, a process called translation invariance. MaxPooling layers allow the creation of a hierarchy of features from edges to objects parts, thus establishing CNNs as the default structure for image classification.

4 Datasets

4.1 SMS Spam Collection (ANN)

- **Name:** SMS Spam Collection Dataset
- **Link:** https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip
- **Type:** Text classification dataset
- **Size:** 5,572 SMS messages - 4,825 ham and 747 spam
- **Suitability:** A publicly available, well-studied benchmark for binary text classification containing real-world English SMS messages with realistic class imbalance (≈13% spam), mirroring production conditions.
- **Preprocessing:**
  - Loaded from a tab-separated file into a pandas DataFrame.
  - Labels encoded: ham → 0, spam → 1.
  - TF-IDF vectorization applied with `max_features=3000` and English stop-word removal, fit only on the training set to prevent data leakage.
  - Split: 80% train (4,457) / 10% validation (557) / 10% test (558), stratified by label.

4.2 MNIST Handwritten Digits (Autoencoder)

- **Name:** MNIST Handwritten Digits Dataset
- **Link:** http://yann.lecun.com/exdb/mnist/ (also available via `tensorflow.keras.datasets.m`
- **Type:** Grayscale image dataset
- **Size:** 70,000 images - 60,000 train / 10,000 test, 28×28 pixels, 10 digit classes
- **Suitability:** This dataset serves as the benchmark image dataset for deep learning applications. The standardisation of the format (white background with centred numbers, standardised dimensions) ensures that the Autoencoder can effectively verify that the reconstruction process is functioning properly.
- **Preprocessing:**
  - Loaded from `keras.datasets.mnist`.
  - Pixel values normalised from [0, 255] to [0.0, 1.0].

- Each 28×28 image flattened to a 784-dimensional vector (for dense layers).
- Labels are *not* used - training is fully unsupervised.

4.3 Fashion-MNIST (CNN)

- **Name:** Fashion-MNIST
- **Link:** https://github.com/zalandoresearch/fashion-mnist (also available via `tensorflow.keras.datasets.fashion_mnist`)
- **Type:** Grayscale image dataset - 10 clothing categories
- **Size:** 70,000 images - 60,000 train / 10,000 test, 28×28 pixels
- **Suitability:** This dataset is designed to be a more challenging substitute for MNIST as it necessitates the ability to discern minor visual differences among classes of clothing items (such as shirts from pullovers).
- **Preprocessing:**
  - Loaded from `keras.datasets.fashion_mnist`.
  - Pixel values normalised from [0, 255] to [0.0, 1.0].
  - Reshaped to (28, 28, 1) to add the channel dimension required by Conv2D.
  - Labels one-hot encoded for categorical crossentropy loss.
  - 60,000 training images split into 54,000 train / 6,000 validation.

5 Model Architectures

5.1 ANN Architecture

`Input(3000)` → `Dense(64, ReLU)` → `Dropout(0.3)` → `Dense(32, ReLU)` → `Dense(1, Sigmoid)`

Table 1: ANN Layer Summary

Layer	Units	Activation	Purpose
Input	3,000	-	TF-IDF feature vector
Dense	64	ReLU	Learn non-linear feature combinations
Dropout	0.3 rate	-	Regularization
Dense	32	ReLU	Further abstraction
Dense (output)	1	Sigmoid	Spam probability (0-1)

Loss: Binary Crossentropy **Optimizer:** Adam **Epochs:** 20 **Batch size:** 32

Justification: For binary classification, binary cross entropy is used as the loss function. Adaptive learning rates are provided by Adam. Dropout(0.3) helps avoid overfitting to the small dataset. The sigmoid function outputs the probability from the logit. If $p > 0.5$, the mail is marked as spam.

5.2 Autoencoder Architecture

Encoder:

`Input(784)` → `Dense(128, ReLU)` → `Dense(64, ReLU)` → `Dense(32, ReLU)`

Decoder:

`Dense(32)` → `Dense(64, ReLU)` → `Dense(128, ReLU)` → `Dense(784, Sigmoid)`

Table 2: Autoencoder Layer Summary

Part	Layer	Units	Activation	Purpose
Encoder	Dense	128	ReLU	Initial compression
Encoder	Dense	64	ReLU	Further compression
Encoder (latent)	Dense	32	ReLU	Bottleneck (latent space)
Decoder	Dense	64	ReLU	Expansion
Decoder	Dense	128	ReLU	Further expansion
Decoder (output)	Dense	784	Sigmoid	Reconstructed image

Loss: Mean Squared Error (MSE) **Optimizer:** Adam (lr=0.001) **Epochs:** 30 **Batch size:** 256

Justification: The bottleneck layer (784 → 32, 24x compression) ensures that the encoder extracts only the key structure from every digit. ReLU avoids vanishing gradients in the hidden layer. The sigmoid activation in the output layer keeps pixel values between [0, 1], matching the scaled inputs. MSE gives more weight to large pixel errors.

5.3 CNN Architecture

`Input(28,28,1)` → `Conv2D(32,3×3,ReLU)` → `MaxPool(2×2)` → `Conv2D(64,3×3,ReLU)` → `MaxPool(2×2)` → `Flatten` → `Dense(128,ReLU)` → `Dropout(0.5)` → `Dense(10,Softmax)`

Table 3: CNN Layer Summary

Layer	**Filters / Units**	**Activation**	**Purpose**
Conv2D	32 filters, 3×3	ReLU	Low-level feature extraction
MaxPooling2D	2×2	-	Spatial downsampling
Conv2D	64 filters, 3×3	ReLU	Higher-level feature extraction
MaxPooling2D	2×2	-	Spatial downsampling
Flatten	-	-	Vectorise feature maps
Dense	128	ReLU	Fully connected reasoning
Dropout	0.5 rate	-	Regularization
Dense (output)	10	Softmax	Class probabilities

Loss: Categorical Crossentropy **Optimizer:** Adam **Epochs:** 20 **Batch size:** 64

Justification: Convolutional blocks increasingly extract edge-detectors to more abstract patterns. The operation of max-pooling decreases dimensions as well as providing invariance to translation. Dense(128) fuses spatial information for classification purposes. Dropout(0.5) after dense layer mitigates over-fitting. Softmax provides probabilities across 10 classes.

6 Training Process

6.1 ANN Training

Data preparation: The SMS Spam Collection data was imported, and the labels were binary encoded. Term Frequency-Inverse Document Frequency (TF-IDF) vectorization (3,000 features, with English stop-words filtered) was trained only on the training dataset in order to avoid information leakage. This same vectorizer vocabulary was used for validation and test datasets.

Training: The model training was done for 20 epochs using a batch size of 32, employing Adam optimisation technique and binary crossentropy loss function. Validation split was employed throughout the training process to check for overfitting.

Overfitting: Both training and validation loss decreased consistently across all 20 epochs, indicating successful learning without significant overfitting.

6.2 Autoencoder Training

Data preprocessing: MNIST dataset was imported using the built-in Keras split (60K training, 10K testing). The pixel intensities were scaled between [0, 1] and all the images were reshaped to be represented in 784 dimensions. No use of labels.

Training: The Autoencoder model was trained for 30 epochs with a batch size of 256, the optimizer used was Adam (learning rate = 0.001) and the loss function was Mean Squared Error. Inputs and outputs were the same images in a normalized format so that the model learned to recreate its inputs.

Overfitting: Loss function on the training set and testing set decreases with time. The bottleneck architecture inherently avoids over-fitting problems.

6.3 CNN Training

Data preparation: Fashion-MNIST was loaded via Keras. Pixel values were normalisedto [0, 1] and reshaped to (28, 28, 1). Labels were one-hot encoded. The 60,000 trainingimages were split into 54,000 training and 6,000 validation samples.

Training: The CNN was trained for 20 epochs with a batch size of 64, using theAdam optimiser and categorical crossentropy loss.

Overfitting: Training and validation accuracy and loss were tracked. Both improvedover 20 epochs. The Dropout(0.5) layer limited overfitting; the gap between training andvalidation accuracy remained modest.

7 Experimental Results

7.1 ANN - SMS Spam Detection

Table 4: ANN Overall Test Metrics

Metric	Value
Test Loss (BCE)	0.1539
Test Accuracy	97.49%

Table 5: ANN Per-Class Classification Report

Class	Precision	Recall	F1-Score	Support
Ham (0)	0.98	0.99	0.99	483
Spam (1)	0.94	0.87	0.90	75
Macro avg	0.96	0.93	0.94	558

Interpretation: The ANN correctly classified 97.49% of test messages. For spam messages (the minority class) it achieved a precision of 0.94 and a recall of 0.87, meaning it missed approximately 13% of actual spam. The high precision minimises false positives (legitimate messages incorrectly flagged as spam). The class imbalance (≈87% ham) makes the spam recall the hardest metric to optimise.

7.2 Autoencoder - MNIST Reconstruction

Table 6: Autoencoder Test Reconstruction Metrics

Metric	Value
Test Reconstruction MSE	0.008680
Test Reconstruction RMSE	0.093167

Interpretation: The Autoencoder achieved a test MSE of 0.0087, meaning on average each reconstructed pixel deviates from the original by ≈0.093 (RMSE) on the [0, 1] scale - roughly 9.3% of the full pixel range. Visually, reconstructed digits are clearly recognisable, demonstrating that the 32-dimensional latent space has captured the essential structural features of each digit. Some smoothing and blurring is expected, as the bottleneck cannot retain all fine-grained pixel details.

7.3 CNN - Fashion-MNIST Classification

Table 7: CNN Overall Test Metrics

Metric	**Value**
Test Accuracy	91%

Table 8: CNN Per-Class Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
T-shirt/top	0.87	0.86	0.87	1,000
Trouser	0.99	0.98	0.99	1,000
Pullover	0.86	0.86	0.86	1,000
Dress	0.92	0.90	0.91	1,000
Coat	0.80	0.91	0.85	1,000
Sandal	0.98	0.99	0.98	1,000
Shirt	0.77	0.69	0.73	1,000
Sneaker	0.96	0.96	0.96	1,000
Bag	0.97	0.98	0.98	1,000
Ankle boot	0.97	0.96	0.97	1,000
Macro avg	0.91	0.91	0.91	10,000

Interpretation: The CNN correctly classified 91% of Fashion-MNIST test images. The easiest classes were Trouser (F1=0.99) and Sandal (F1=0.98), which have distinctive visual shapes. The hardest class was Shirt (F1=0.73), which is visually similar to T-shirt/top, Pullover, and Coat, leading to more confusion. The balanced support (1,000 images per class) makes macro-average a reliable summary: macro F1 = 0.91.

8 Graphs and Commentary

Note: Insert the corresponding screenshots/graphs from your Jupyter notebooks below each subsection. Each figure should include a brief caption.

8.1 ANN - Training Curves

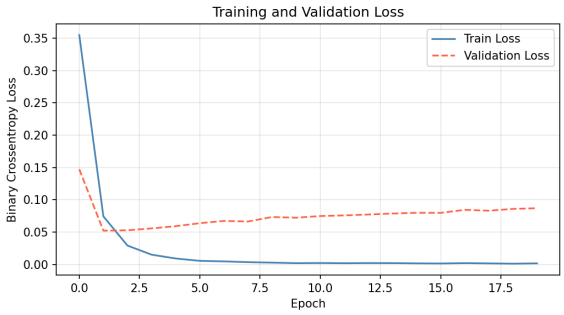

Figure 1: ANN training and validation loss over 20 epochs. Both curves decrease consistently, confirming successful learning without overfitting.

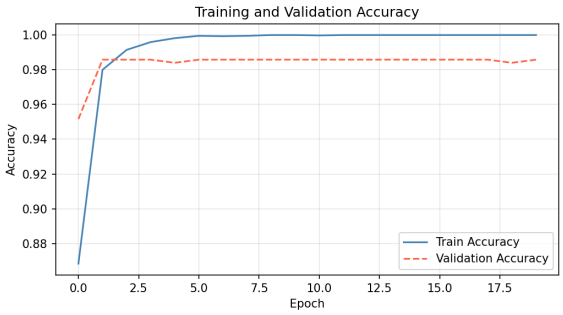

Figure 2: ANN training and validation accuracy over 20 epochs. Both curves converge above 97%, indicating strong generalisation.

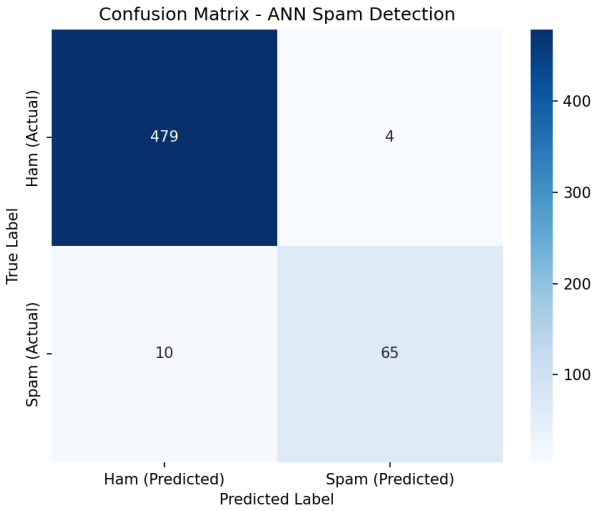

Figure 3: Confusion matrix for ANN on the test set. The model misclassifies 10 spam messages as ham (false negatives) and 4 ham messages as spam (false positives).

8.2 Autoencoder - Reconstruction Results

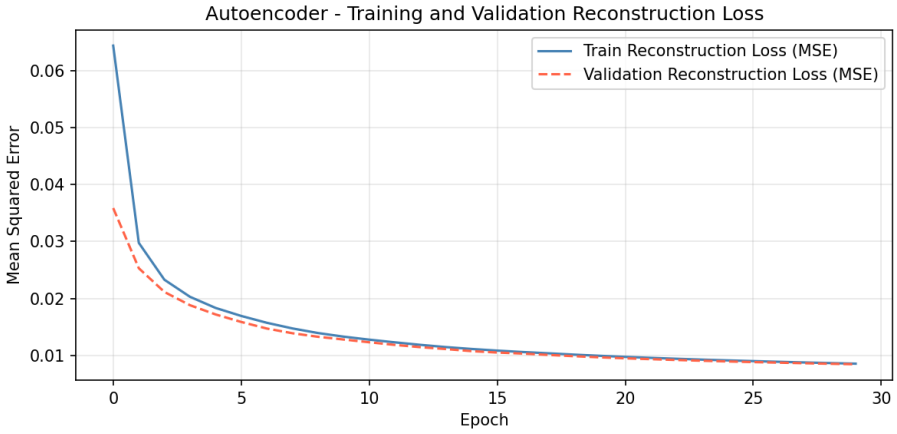

Figure 4: Autoencoder MSE reconstruction loss over 30 epochs. Both train and test loss decrease and converge, demonstrating stable learning.

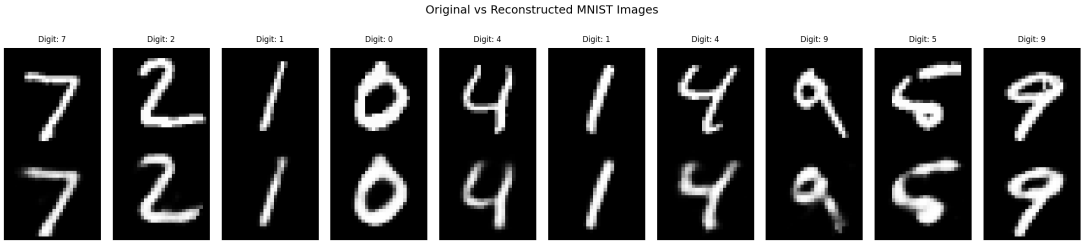

Figure 5: MNIST original images (top row) vs. Autoencoder reconstructions (bottom row). Digits are clearly recognisable; mild blurring is inherent to the 32-dimensional bottleneck.

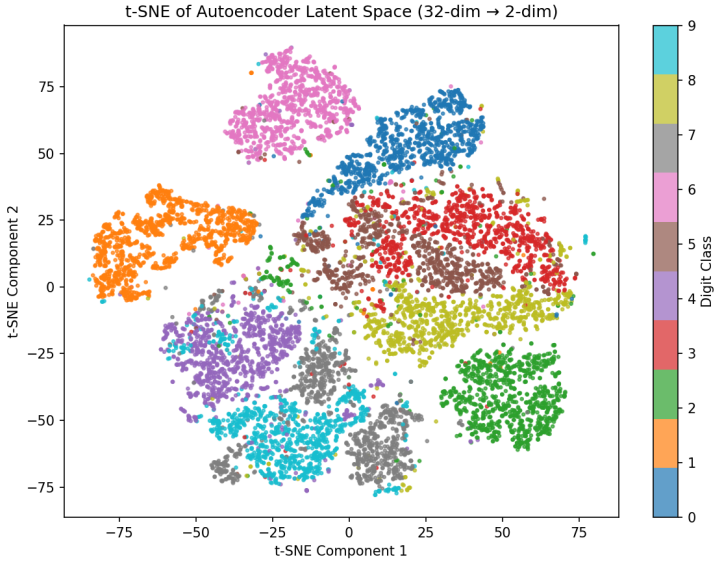

Figure 6: t-SNE visualisation of the 32-dimensional latent space, coloured by digit class. Digit clusters are well-separated, showing the Autoencoder has learned class-discriminative representations despite being trained without labels.

8.3 CNN - Training Curves and Evaluation

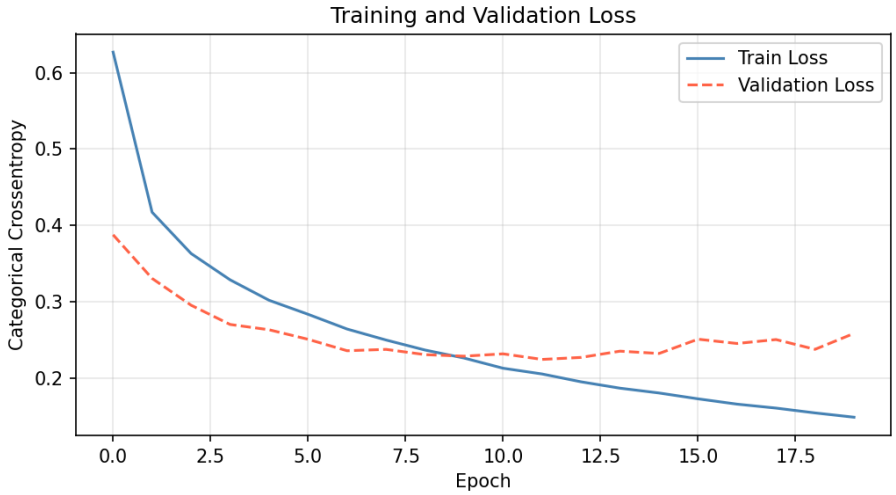

Figure 7: CNN training and validation loss over 20 epochs (Categorical Crossentropy). Loss decreases consistently; the small gap between train and validation loss indicates minimal overfitting.

Figure 8: CNN training and validation accuracy over 20 epochs. Validation accuracy reaches ≈92%, closely tracking training accuracy.

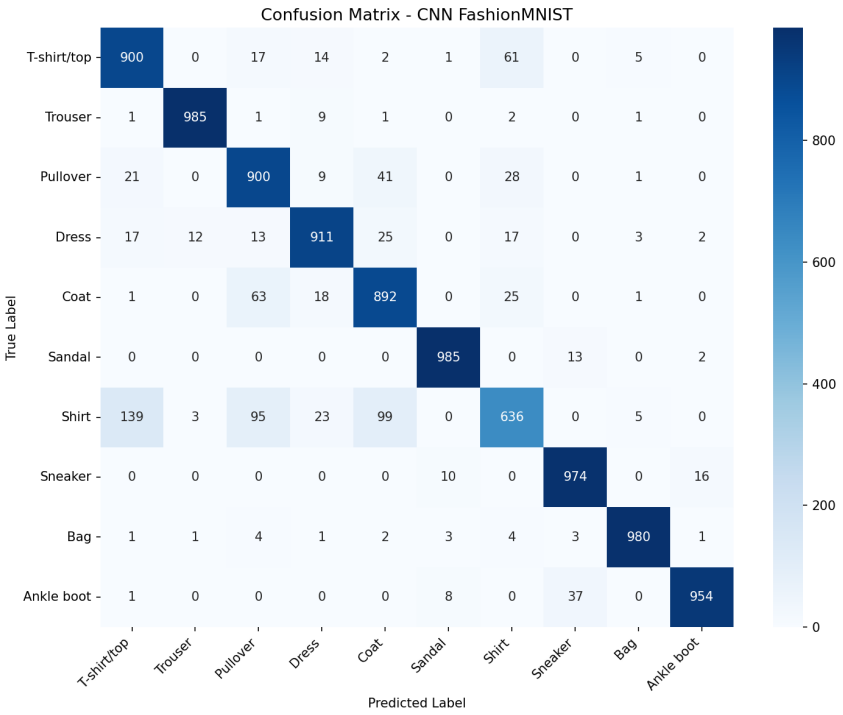

Figure 9: Confusion matrix for CNN on the Fashion-MNIST test set (10,000 images). The highest confusion is between Shirt, T-shirt/top, and Pullover - visually similar categories.

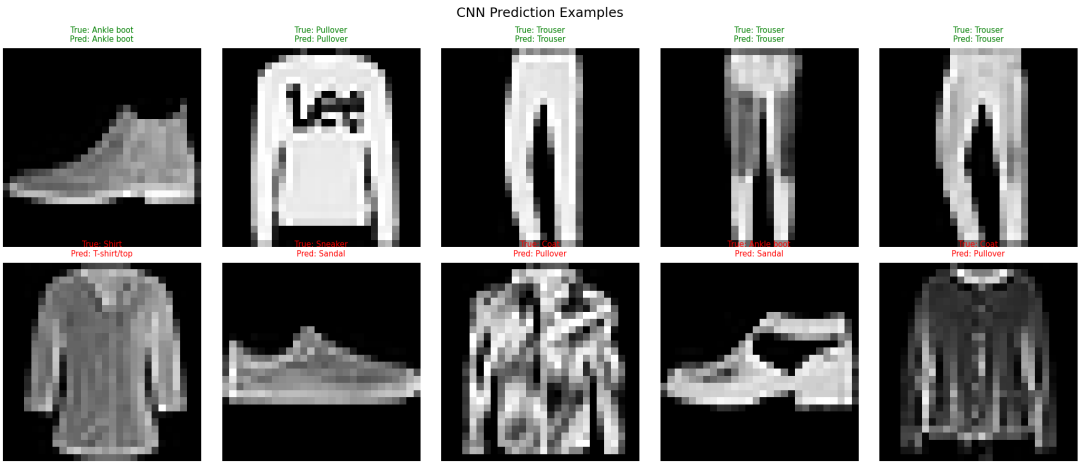

Figure 10: Sample CNN predictions on Fashion-MNIST test images. Green titles indicate correct predictions; red titles indicate misclassifications.

9 Discussion

9.1 Was the model successful?

ANN: Yes. A test accuracy of 97.49% is strong for a binary classification task on an imbalanced dataset. The spam recall of 0.87 means 13% of actual spam messages were missed, which is acceptable for a simple dense network without specialised handling of class imbalance.

Autoencoder: Yes. An RMSE of 0.093 on the [0, 1] pixel scale means the average reconstruction error is below 10% of the full dynamic range. Visual inspection confirms that all 10 digit classes are clearly preserved in the reconstructed output. **CNN:** Yes. A test accuracy of 91% on the Fashion-MNIST benchmark (10 classes, balanced) is solid performance for a compact two-block CNN. The macro-average F1 of 0.91 confirms this applies across all classes, not just easy ones.

9.2 Did loss decrease during training?

In all three models, both training and validation loss decreased consistently across epochs, confirming that gradient-based optimisation was effective and that the models were learning.

9.3 Was validation performance close to training performance?

ANN: Yes. The small gap between training and validation curves confirms minimal overfitting, aided by Dropout(0.3). **Autoencoder:** Yes. Train and test MSE tracked closely, suggesting the bottleneck provides sufficient implicit regularisation. **CNN:** Yes. The train–validation accuracy gap remained small throughout training, largely due to Dropout(0.5).

9.4 Was overfitting observed?

No significant overfitting was observed in any model. Dropout regularisation was the main preventive mechanism used.

9.5 Strengths and Weaknesses

Table 9: Strengths and Weaknesses Summary

Model	Strengths	Weaknesses
ANN (Spam)	Fast training; strong ham detection; no sequential input needed	Spam recall limited by class imbalance; no context or word order
Autoencoder	Fully unsupervised; learns compressed representations; stable training	Dense layers cannot exploit spatial structure; some blurring inevitable
CNN (Fashion)	Excellent spatial feature learning; strong across most classes	Shirt class confusion with visually similar tops; no data augmentation

9.6 What could be improved?

- **ANN:** Class weighting and oversampling techniques (SMOTE) will be applied to help improve spam recall rate. Try out more extensive vocabularies (5,000-10,000 TF-IDF features) or use character-level n-gram representations to model morphological patterns in spam emails.
- **Autoencoder:** Switch to Convolutional Autoencoder that will allow to utilize spatial structure in images to produce clearer reconstructions. Increasing the dimensionality of latent space (e.g., to 64) will eliminate blurring but make the compression ratio worse.
- **CNN:** Augment data set through random flipping, rotating, and zooming to further prevent overfitting. Increase network depth (third block of convolutions) or apply batch normalization to boost performance above 92-93%.

10 Conclusion

In this project, three different deep learning architectures were developed and evaluated, including both text and image datasets, and supervised and unsupervised learning:

1. ANN to detect SMS spam messages reached an accuracy of 97.49%, proving that even using a simple feed-forward architecture, it is possible to perform high-quality binary text classification using only TF-IDF features.

2. Dense autoencoder for image reconstruction task on the MNIST dataset reached a test error of 0.0087 (RMSE: 0.093), generating meaningful images using 32-D latent variables and achieving no degradation in the quality of reconstructions despite using no labels at all.

3. CNN for classification task on the Fashion-MNIST dataset reached accuracy of 91%, confirming that convolutional neural networks with hierarchical pooling can accurately extract meaningful patterns to differentiate between 10 types of fashion products.

It is noteworthy that in none of the cases was there any indication of overfitting; training and validation errors were tracked closely during training. The future development involves class weight balancing for the ANN, adding convolutional layers to the Autoencoder and data augmentation for CNNs.

References

- Abadi, M., et al. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. Software available at https://www.tensorflow.org/.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://scikit-learn.org/
- Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://numpy.org/
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. https://matplotlib.org/
- Waskom, M. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. https://seaborn.pydata.org/
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56. https://pandas.pydata.org/
- Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. *Proceedings of the 11th ACM Symposium on Document Engineering*, 259–262. Dataset: https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip
- LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). The MNIST database of handwritten digits. http://yann.lecun.com/exdb/mnist/
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*. Dataset: https://github.com/zalandoresearch/fashion-mnist